

The `git stash` command temporarily shelves (stores) uncommitted modifications in your working directory so you can work on something else, leaving you with a clean workspace. It is widely used to switch branches or pull upstream updates without committing half-finished work.  W3Schools +3

Core Commands

1. Saving Changes

- `git stash` or `git stash push`: Saves your modified tracked files and reverts the workspace to match the `HEAD` commit.
- `git stash push -m "your message"`: Saves the stash with a descriptive label for easier tracking.
- `git stash -u` (or `--include-untracked`): Stashes untracked (newly created) files along with tracked ones.
- `git stash -S` (or `--staged`): Stashes only the modifications that you have currently staged.  W3Schools +5

2. Reviewing Stashes

- `git stash list`: Displays all your saved stashes in a stack-like index format (e.g., `stash@{0}` is the newest).
- `git stash show -p stash@{n}`: Shows the detailed code diff of a specific stash entry.  Git +4

3. Restoring Changes

- `git stash pop`: Applies the most recent stash entry (`stash@{0}`) back to your workspace and immediately deletes it from the stack.
- `git stash apply`: Re-applies the most recent stash but keeps it saved in the stash list.
- `git stash apply stash@{n}`: Restores a specific older stash from your list.  YouTube · Colt Steele +4

4. Cleaning Up

- `git stash drop stash@{n}`: Deletes a specific stash entry permanently.
- `git stash clear`: Wipes out all stored stashes across the active repository.  GeeksforGeeks +1

Comparison: Pop vs. Apply

Feature 	<code>git stash pop</code>	<code>git stash apply</code>
Workspace Effect	Restores the uncommitted code.	Restores the uncommitted code.
Stack History	Deletes the stash entry from the list.	Keeps the stash entry in the list.
Best Used For	Quick context switches.	Testing changes across multiple branches.

For deep workflow scenarios or step-by-step edge cases, review the official [git-stash Documentation](#). 